

РЭУ.РФ
РОССИЙСКИЙ ЭКОНОМИЧЕСКИЙ УНИВЕРСИТЕТ
ИМЕНИ Г.В. ПЛЕХАНОВА

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Пермский институт (филиал)

Отчет

по учебной практике

УП.02.01 Технология разработки программного обеспечения
(индекс по РУП и наименование производственной практики)

Профессионального модуля ПМ.02 Осуществление интеграции
программных модулей

Специальность 09.02.07 «Информационные системы и программирование»

Обучающийся _____ Светлышева Елизавета Денисовна
(подпись) (фамилия, имя, отчество)

Группы ИПо-32

Руководитель практической подготовки от профильной организации

Преподаватель
(должность)

_____ Берестов Дмитрий Борисович
(подпись) (фамилия, имя, отчество)

МП «26» июня 2026 года

Руководитель практической подготовки от института

_____ Берестов Дмитрий Борисович
(подпись) (фамилия, имя, отчество)

«27» июня 2026 года

Пермь, 2026 год

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	3
РАЗДЕЛ 1. РАЗРАБОТКА ТРЕБОВАНИЯ К ПРОГРАММНЫМ МОДУЛЯМ НА ОСНОВЕ АНАЛИЗА ПРОЕКТНОЙ И ТЕХНИЧЕСКОЙ ДОКУМЕНТАЦИИ НА ПРЕДМЕТ ВЗАИМОДЕЙСТВИЯ КОМПОНЕНТ	5
1.1. Сбор информации о существующем состоянии продукта	5
1.2. Разработка технического задания	7
РАЗДЕЛ 2. ИНТЕГРАЦИЯ МОДУЛЕЙ В ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ	10
2.1. Процесс разработки программного обеспечения	10
2.2. Выбор модели разработки	10
2.3. Инструменты разработки.....	12
2.4. Диаграмма классов	13
2.5. Структура приложения	14
2.6. Этапы разработки ПО	16
2.7. Разработка форм и интерфейса	17
РАЗДЕЛ 3. ОТЛАДКА ПРОГРАММНОГО МОДУЛЯ С ИСПОЛЬЗОВАНИЕМ СПЕЦИАЛИЗИРОВАННЫХ ПРОГРАММНЫХ СРЕДСТВ.....	20
3.1. Процесс отладки	20
РАЗДЕЛ 4. ОСУЩЕСТВЛЯТЬ РАЗРАБОТКУ ТЕСТОВЫХ НАБОРОВ И ТЕСТОВЫХ СЦЕНАРИЕВ ДЛЯ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ	22
4.1. Тестирование.....	22
4.2. Unit-тесты	24
РАЗДЕЛ 5. ИНСПЕКТИРОВАНИЕ КОМПОНЕНТ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ НА ПРЕДМЕТ СООТВЕТСТВИЯ СТАНДАРТАМ КОДИРОВАНИЯ	25
5.1. Процесс инспектирования компонент программного обеспечения на предмет соответствия стандартам кодирования.....	25
ЗАКЛЮЧЕНИЕ.....	28
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ	30

ВВЕДЕНИЕ

Практика имеет целью комплексное освоение студентами всех видов профессиональной деятельности по специальности 09.02.07

«Информационные системы и программирование», формирование общих и профессиональных компетенций, а также приобретение необходимых умений и опыта практической работы по специальности.

Учебная практика направлена на формирование у обучающихся первоначальных практических профессиональных умений в рамках модулей ППССЗ по основным видам профессиональной деятельности для освоения методов и приемов практического применения прикладных программных продуктов для программного обеспечения компьютерных систем

В настоящей работе представлен отчет о прохождении учебной практики студента-практиканта. Основными видами деятельности являлись: Разработка программного обеспечения, выполнение отладки программного модуля с использованием специализированных программных средств, осуществление разработки тестовых наборов и тестовых сценариев для программного обеспечения, уметь производить инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования

Задачи практики (освоение обучающимися профессиональных и общих компетенций):

1. Разрабатывать требования к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент.
2. Выполнять интеграцию модулей в программное обеспечение.
3. Выполнять отладку программного модуля с использованием специализированных программных средств.

4. Осуществлять разработку тестовых наборов и тестовых сценариев для программного обеспечения.
5. Производить инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования.
6. Выбирать способы решения задач профессиональной деятельности, применительно к различным контекстам.
7. Осуществлять поиск, анализ и интерпретацию информации, необходимой для выполнения задач профессиональной деятельности.
8. Планировать и реализовывать собственное профессиональное и личностное развитие.
9. Работать в коллективе и команде, эффективно взаимодействовать с коллегами, руководством, клиентами.
10. Осуществлять устную и письменную коммуникацию на государственном языке Российской Федерации с учетом особенностей социального и культурного контекста.
11. Использовать информационные технологии в профессиональной деятельности.
12. Пользоваться профессиональной документацией на государственном и иностранном языках.

РАЗДЕЛ 1. РАЗРАБОТКА ТРЕБОВАНИЯ К ПРОГРАММНЫМ МОДУЛЯМ НА ОСНОВЕ АНАЛИЗА ПРОЕКТНОЙ И ТЕХНИЧЕСКОЙ ДОКУМЕНТАЦИИ НА ПРЕДМЕТ ВЗАИМОДЕЙСТВИЯ КОМПОНЕНТ

1.1. Сбор информации о существующем состоянии продукта

Постановка задачи:

1. Целью практики является разработка приложения для управления задачами с использованием языка программирования C# и технологии WPF. Приложение должно реализовывать полный набор функций по созданию, просмотру, редактированию, удалению и сохранению задач, предоставляя пользователю удобный графический интерфейс.

2. Описание бизнес-процесса «AS IS» (до автоматизации)

До внедрения системы управления задачами пользователи вынуждены хранить списки дел в текстовых редакторах, электронных таблицах или бумажных носителях. Основные проблемы данного подхода:

- отсутствие единой структуры хранения задач - каждый пользователь ведет собственный формат записей.
- невозможность быстрой фильтрации и поиска задач по различным критериям (статус, приоритет, срок выполнения).
- сложность контроля просроченных задач - необходимо вручную сверять даты выполнения.
- отсутствие статистики по выполнению задач.
- высокий риск потери данных при использовании локальных файлов без резервного копирования.

3. Описание процесса модернизации «TO BE» (после автоматизации)

После внедрения приложения TaskManager процесс управления задачами приобретает следующий вид: пользователь запускает WPF-приложение, которое автоматически загружает сохраненный список задач из JSON-файла.

Далее доступны следующие операции:

- добавление новой задачи с указанием названия, описания, приоритета, срока выполнения и статуса.
- просмотр всех задач в виде структурированного списка с визуальным выделением важных и просроченных задач.
- фильтрация задач по статусу (Новая / В процессе / Завершена) в режиме реального времени.
- полнотекстовый поиск по названию и описанию задачи.
- сортировка задач по приоритету или сроку выполнения.
- редактирование существующих задач через отдельное диалоговое окно.
- удаление задач с подтверждением операции.
- просмотр статистики (всего задач, завершено, просрочено, важных).
- сохранение и загрузка задач из JSON-файла по выбору пользователя.

4. Сравнительный анализ процессов «AS IS» и «TO BE»

Ниже представлена Таблица 1 - Сравнительный анализ процессов, которая рассказывает о управления задачами до и после автоматизации «AS IS» и «TO BE».

Таблица 1 - Сравнительный анализ процессов

Критерий	AS IS (до автоматизации)	TO BE (после автоматизации)
Хранение данных	Текстовые файлы, бумажные носители	JSON-файл с автоматической загрузкой
Поиск задачи	Ручной просмотр всего списка	Полнотекстовый поиск по названию и описанию
Фильтрация	Отсутствует	По статусу, приоритету, сроку
Контроль просрочки	Ручная проверка дат	Автоматическое выделение просроченных задач
Статистика	Отсутствует	Автоматический подсчет по всем категориям
Интерфейс	Текстовый редактор / Excel	Графический WPF-интерфейс
Надежность хранения	Низкая	Высокая (JSON с возможностью резервирования)

1.2. Разработка технического задания

Основные требования к техническому заданию:

Техническое задание разрабатывается в соответствии с ГОСТ 19.201-78 «Техническое задание. Требования к содержанию и оформлению» и ГОСТ 34.602-89 «Техническое задание на создание автоматизированной системы».

Стандарты, используемые для написания технического задания:

– ГОСТ 19 (ЕСПД - Единая система программной документации) - регламентирует состав и требования к программным документам, в том числе техническому заданию на разработку программы и программной документации.

– ГОСТ 34 (комплекс стандартов на автоматизированные системы) - определяет стадии разработки, виды обеспечения и требования к содержанию технического задания на создание автоматизированных систем.

Содержание технического задания.

Техническое задание на разработку приложения TaskManager включает следующие разделы:

- наименование и область применения - приложение для управления списком задач, предназначено для студентов и специалистов, нуждающихся в инструменте планирования.
- основание для разработки - задание на учебную практику УП.02.01.
- назначение разработки - создание программного комплекса, автоматизирующего учет и управление задачами.
- требования к программе - функциональные (добавление, редактирование, удаление, фильтрация, поиск, сортировка, сохранение) и нефункциональные (производительность, надежность, удобство использования).
- требования к программной документации - наличие диаграммы классов, описания структуры и алгоритмов работы.
- технико-экономические показатели - оценка трудозатрат на реализацию.
- стадии и этапы разработки - проектирование, кодирование, тестирование, документирование.
- порядок контроля и приемки - модульное тестирование, функциональное тестирование, сдача руководителю практики.

Структура технического задания:

В соответствии с ГОСТ 19.201-78 техническое задание оформляется в виде документа, содержащего титульный лист, содержание, основную часть с разделами согласно стандарту, а также приложения (при необходимости).

Каждое требование формулируется однозначно и проверяемо.

Порядок документирования требований.

Документирование требований осуществляется поэтапно: На первом этапе производится сбор требований путем анализа задания на практику. На

втором этапе требования формализуются и разбиваются на функциональные и нефункциональные группы, на третьем этапе требования согласовываются с руководителем практики и фиксируются в техническом задании. Каждое требование имеет уникальный идентификатор для отслеживания его реализации.

РАЗДЕЛ 2. ИНТЕГРАЦИЯ МОДУЛЕЙ В ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ

2.1. Процесс разработки программного обеспечения

Разработка приложения TaskManager велась поэтапно в соответствии с выбранной моделью разработки. Процесс включал в себя проектирование архитектуры, реализацию бизнес-логики в динамической библиотеке (TaskManager.Core), создание пользовательского интерфейса на платформе WPF (TaskManager.WPF), написание модульных тестов (TaskManager.Tests) и отладку готового приложения.

2.2. Выбор модели разработки

Для разработки приложения TaskManager была выбрана итерационная модель разработки программного обеспечения. Данный выбор обусловлен следующими причинами:

- приложение разрабатывается одним исполнителем в условиях ограниченных временных ресурсов, что позволяет использовать короткие итерации с быстрой обратной связью.
- требования к приложению в целом известны, однако в процессе реализации могут уточняться - итерационный подход позволяет вносить изменения без серьезных последствий для всего проекта.
- каждая итерация завершается рабочей версией приложения, что позволяет проверять корректность реализованных функций до перехода к следующей итерации.

Рассмотрим основные модели разработки ПО и обоснование выбора в Таблица 2 – Модели разработки ПО и обоснование выбора:

Таблица 2 – Модели разработки ПО и обоснование выбора

Модель	Описание	Применимость к проекту
Каскадная (водопадная)	Последовательные фазы: требования → проектирование → реализация → тестирование → сопровождение	Не подходит - требует полной фиксации требований до начала разработки
V-образная модель	Расширение каскадной модели с акцентом на верификацию и валидацию на каждом этапе	Частично применима, но избыточна для учебного проекта
Модель прототипирования	Создание прототипа для уточнения требований	Применима на этапе проектирования интерфейса
RAD-модель	Быстрая разработка с активным участием пользователей	Не применима - проект выполняется без заказчика
Итерационная модель	Разработка ведется в итерациях, каждая из которых добавляет функциональность	Выбрана - наилучшее соответствие условиям проекта
Спиральная модель	Акцент на управлении рисками, повторяющиеся циклы	Избыточна для проекта данного масштаба

В рамках итерационной модели разработка приложения TaskManager осуществлялась в три итерации:

– Итерация 1: проектирование архитектуры, разработка моделей данных (TaskItem, TaskStatistics), реализация перечислений TaskStatus и TaskPriority.

– Итерация 2: реализация интерфейсов ITaskRepository и ITaskService, разработка классов JsonTaskRepository и TaskService, реализация бизнес-логики с применением LINQ.

– Итерация 3: разработка пользовательского интерфейса на WPF с применением паттерна MVVM, написание модульных тестов, отладка и финальное тестирование приложения.

2.3. Инструменты разработки

Для создания приложения TaskManager использовались следующие инструменты и технологии их можно увидеть в Таблица 3 – Инструменты и технологии:

Таблица 3 – Инструменты и технологии.

Инструмент / Технология	Назначение	Версия
Microsoft Visual Studio 2022	Интегрированная среда разработки (IDE)	17.x
C# 12	Язык программирования	12.0
.NET 8	Платформа выполнения приложений	8.0 LTS
WPF (Windows Presentation Foundation)	Технология создания графического интерфейса	.NET 8
MVVM Pattern	Архитектурный паттерн для WPF-приложений	—
LINQ (Language Integrated Query)	Запросы к коллекциям данных	—
System.Text.Json	Сериализация/десериализация данных в JSON	—
xUnit	Фреймворк для написания модульных тестов	2.x
Git / GitHub	Система контроля версий	—

Описание среды программирования:

Microsoft Visual Studio 2022 - это полнофункциональная интегрированная среда разработки (IDE) от компании Microsoft, предоставляющая полный набор инструментов для разработки, отладки и тестирования приложений на платформе .NET. Среда включает встроенный дизайнер WPF-форм, инструменты рефакторинга, инспектор переменных в режиме отладки, встроенный терминал, систему управления пакетами NuGet, а также интеграцию с системой контроля версий Git.

Решение (Solution) TaskManager включает три проекта: TaskManager.Core (библиотека классов, содержащая всю бизнес-логику), TaskManager.WPF (WPF-приложение с пользовательским интерфейсом) и TaskManager.Tests (проект с модульными тестами на базе xUnit).

2.4. Диаграмма классов

Диаграмма классов приложения TaskManager отражает архитектуру программного решения и взаимодействие между его компонентами. В соответствии с принципом разделения ответственности и принципами SOLID, архитектура разделена на несколько уровней смотреть на Рисунок 2.4.1 – Диаграмма классов.

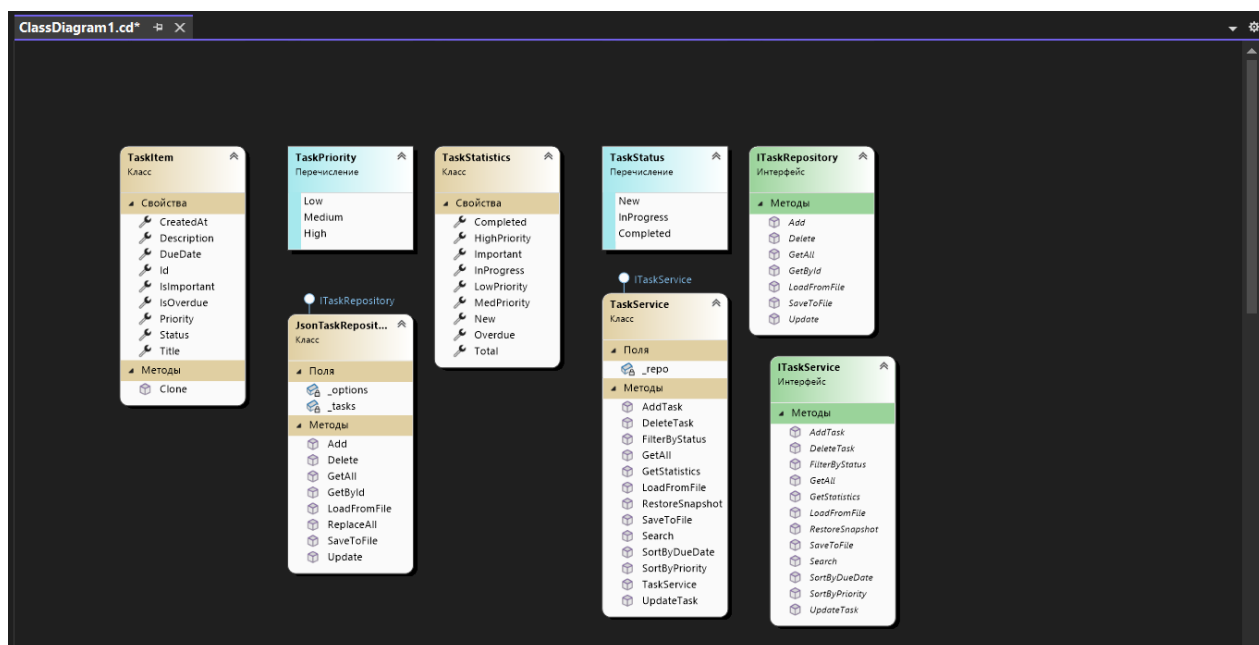


Рисунок 2.4.1 – Диаграмма классов

Диаграмма классов включает следующие элементы:

- TaskItem - основная модель данных, описывающая задачу. Содержит свойства: Id (уникальный идентификатор типа Guid), Title (название), Description (описание), Priority (приоритет типа TaskPriority), DueDate (срок выполнения), Status (статус типа TaskStatus), IsImportant (флаг важности), CreatedAt (дата создания). Вычисляемое свойство IsOverdue определяет, просрочена ли задача. Метод Clone() создает копию объекта.

- TaskStatistics - вспомогательная модель, содержащая агрегированную статистику по задачам: Total, New, InProgress, Completed, Overdue, Important, HighPriority, MedPriority, LowPriority.
- TaskPriority - перечисление (enum), определяющее уровни приоритета: Low (низкий), Medium (средний), High (высокий).
- TaskStatus - перечисление (enum), определяющее статусы задачи: New (новая), InProgress (в процессе), Completed (завершена).
- ITaskRepository - интерфейс репозитория, объявляющий методы GetAll(), GetById(), Add(), Update(), Delete(), SaveToFile() и LoadFromFile(). Обеспечивает абстракцию уровня хранения данных.
- JsonTaskRepository - конкретная реализация интерфейса ITaskRepository, осуществляющая хранение задач в JSON-файле с использованием System.Text.Json. Дополнительно содержит метод ReplaceAll() для восстановления состояния из снимка.
- ITaskService - интерфейс сервисного уровня, объявляющий методы бизнес-логики: GetAll(), FilterByStatus(), Search(), SortByPriority(), SortByDueDate(), GetStatistics(), AddTask(), UpdateTask(), DeleteTask(), SaveToFile(), LoadFromFile(), RestoreSnapshot().
- TaskService - реализация ITaskService, содержащая бизнес-логику приложения с использованием LINQ-запросов.

2.5. Структура приложения

Структура решения TaskManager организована по принципу многоуровневой архитектуры и наглядно представлена в обозревателе решений Visual Studio. Смотреть на Рисунок 2.5.1 – Структура решения TaskManager.

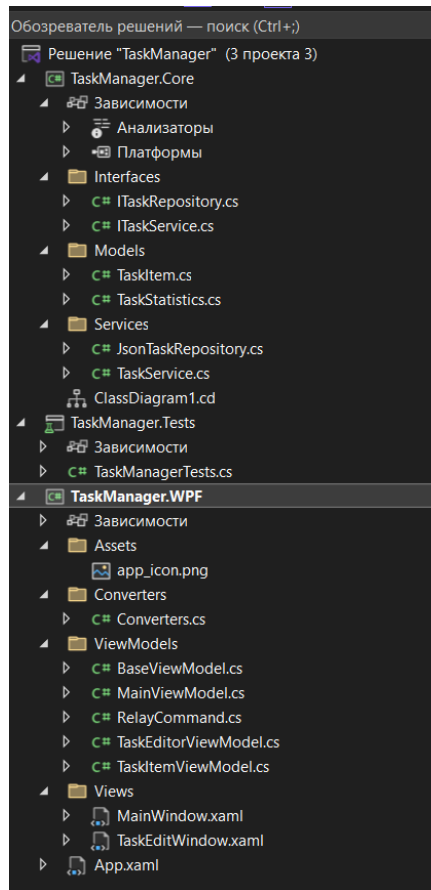


Рисунок 2.5.1 – Структура решения TaskManager

Решение состоит из трёх проектов:

- TaskManager.Core - библиотека классов (.NET 8), реализующая бизнес-логику приложения. Включает: папку Interfaces (ITaskRepository.cs, ITaskService.cs), папку Models (TaskItem.cs, TaskStatistics.cs), папку Services (JsonTaskRepository.cs, TaskService.cs).

- TaskManager.Tests - проект с модульными тестами на базе xUnit. Содержит файл TaskManagerTests.cs с набором из 29 тестов для проверки корректности работы бизнес-логики.

- TaskManager.WPF - WPF-приложение, реализующее пользовательский интерфейс. Включает: папку Assets (иконка приложения), папку Converters (Converters.cs - конвертеры значений для XAML), папку ViewModels (BaseViewModel.cs, MainViewModel.cs, TaskItemViewModel.cs, TaskEditorViewModel.cs, RelayCommand.cs), папку Views (MainWindow.xaml, TaskEditWindow.xaml), файл App.xaml.

2.6. Этапы разработки ПО

Выбор архитектурного шаблона:

На начальном этапе проектирования была выбрана трёхуровневая архитектура с применением паттерна MVVM (Model-View-ViewModel) для уровня представления. MVVM обеспечивает четкое разделение бизнес-логики и интерфейса, упрощает тестирование и поддержку кода.

Составление схемы информационной базы:

Основной структурой данных является класс `TaskItem`, хранящийся в оперативной памяти в виде `List<TaskItem>` и персистируемый в JSON-файле. Для работы с данными реализован паттерн Repository: интерфейс `ITaskRepository` абстрагирует операции CRUD, а `JsonTaskRepository` предоставляет конкретную реализацию с использованием `System.Text.Json`. Сервисный уровень (`TaskService`) реализует бизнес-логику поверх репозитория.

Документирование системы:

В процессе разработки формировалась следующая документация: диаграмма классов (`ClassDiagram1.cd` в Visual Studio), комментарии в коде в формате XML-документации (`/// <summary>`), настоящий отчёт по учебной практике, а также `README.md` с описанием структуры решения, возможностей приложения, инструкцией по запуску и таблицей используемых технологий.

Сопровождение системы:

Сопровождение приложения предполагает возможность расширения функциональности: добавления новых способов хранения данных (XML, база данных SQLite) путем реализации интерфейса `ITaskRepository`, добавления новых видов сортировки и фильтрации в `TaskService`, а также расширения пользовательского интерфейса без изменения бизнес-логики благодаря архитектуре MVVM.

2.7. Разработка форм и интерфейса

Проектирование интерфейса.

На этапе проектирования интерфейса были определены основные экраны приложения и их функциональное назначение:

– Главное окно (MainWindow) - основной рабочий экран приложения, содержащий панель инструментов с кнопками действий, поле поиска, выпадающий список фильтрации по статусу, список задач с визуальным выделением важных и просроченных задач, панель статистики.

– Диалоговое окно редактирования (TaskEditWindow) - модальное окно для создания и редактирования задачи, содержащее поля ввода: название, описание, приоритет (выпадающий список), дата выполнения (DatePicker), статус (выпадающий список), флаг важности (CheckBox), кнопки «Сохранить» и «Отмена».

Все это можно увидеть на Рисунок – 2.7.1 – Рисунок – 2.7.6

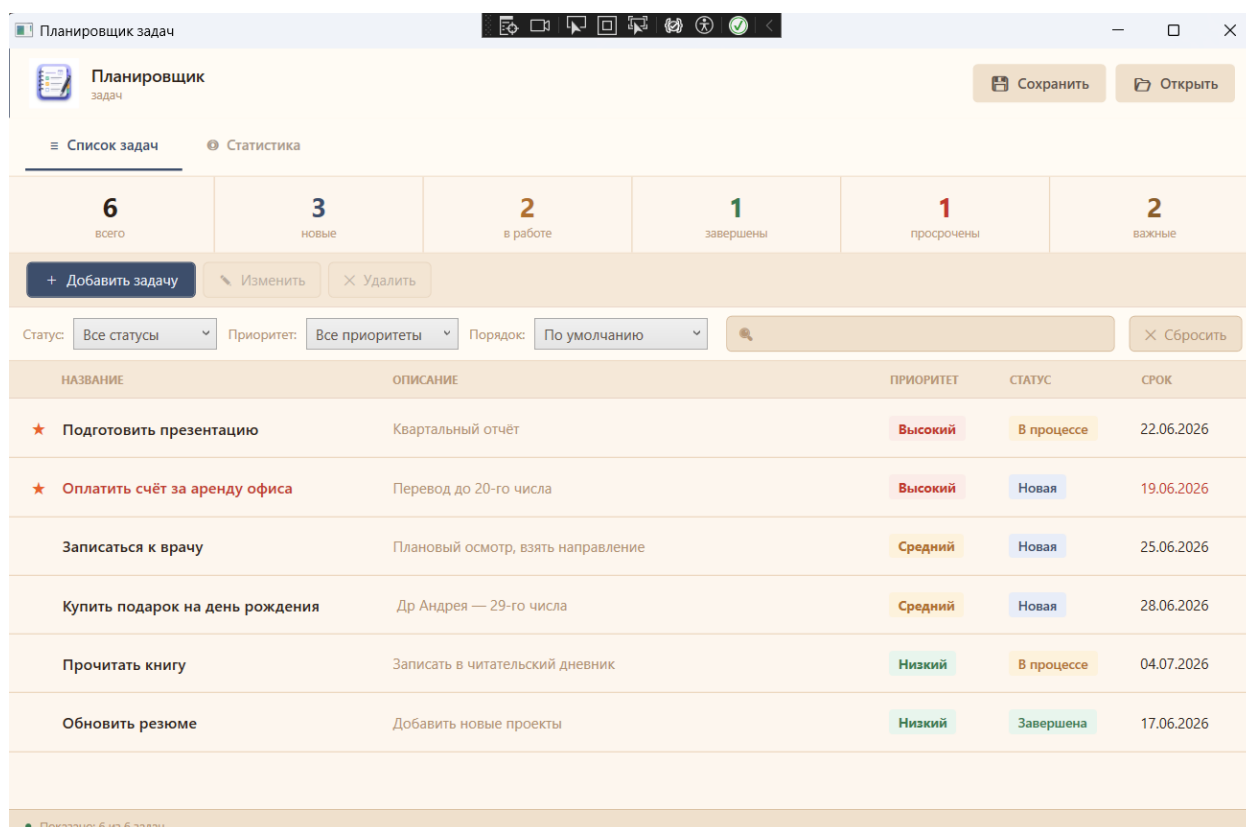


Рисунок 2.7.1 – Главное окно приложения с основными функциями интерфейса

Новая задача

НАЗВАНИЕ *

ОПИСАНИЕ

ПРИОРИТЕТ Средний

СТАТУС Новая

СРОК ВЫПОЛНЕНИЯ 26.06.2026

☐ ★ Важная задача

Отмена Сохранить

Рисунок 2.7.2 – Окно новой задачи

Изменить задачу

НАЗВАНИЕ *

Обновить резюме

ОПИСАНИЕ

Добавить новые проекты и навыки

ПРИОРИТЕТ Низкий

СТАТУС Завершена

СРОК ВЫПОЛНЕНИЯ 17.06.2026

☐ ★ Важная задача

Отмена Сохранить

Рисунок 2.7.3 – Окно изменения задач

Подтверждение

Удалить задачу «Обновить резюме»?

Да Нет

Рисунок 2.7.4 – Окно подтверждения удаления

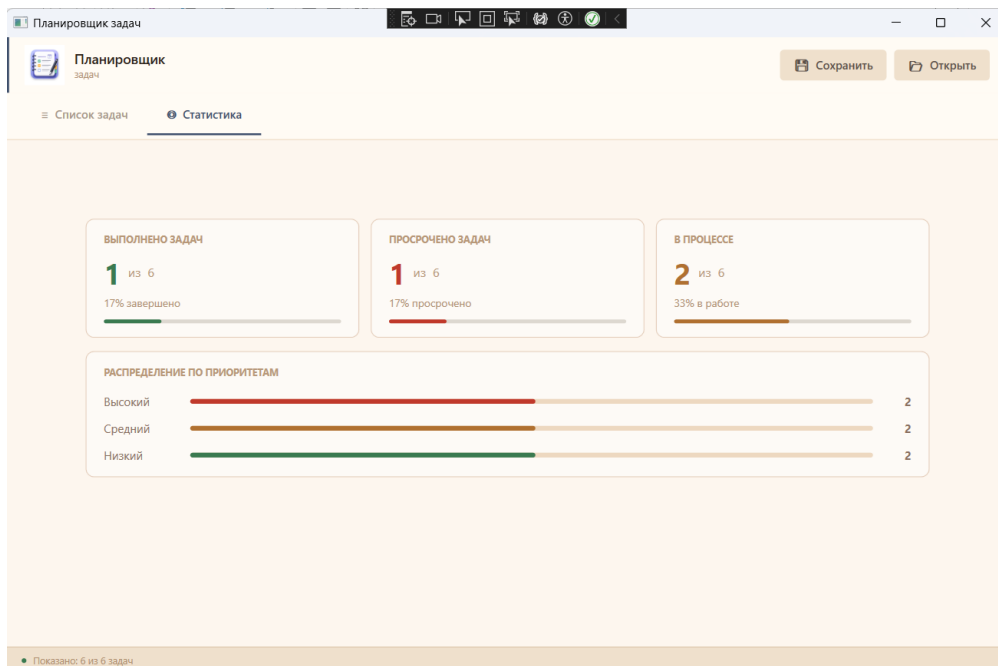


Рисунок 2.7.5 – Вкладка статистики

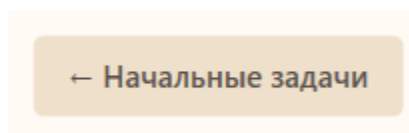


Рисунок 2.7.6 – Кнопка возвращения

Стилизация:

Стилизация интерфейса осуществлялась средствами WPF: в секциях Resources файлов MainWindow.xaml и TaskEditWindow.xaml определены стили для элементов управления (кнопок, выпадающих списков, строк списка задач). Просроченные задачи выделяются красным цветом через XAML-конвертер (OverdueBrushConverter), важные задачи маркируются символом звезды. Для привязки данных использовался механизм Data Binding платформы WPF с реализацией интерфейса INotifyPropertyChanged в базовом классе BaseViewModel.

РАЗДЕЛ 3. ОТЛАДКА ПРОГРАММНОГО МОДУЛЯ С ИСПОЛЬЗОВАНИЕМ СПЕЦИАЛИЗИРОВАННЫХ ПРО- ГРАММНЫХ СРЕДСТВ

3.1. Процесс отладки

Отладка программного обеспечения - процесс обнаружения и устранения ошибок (дефектов) в программном коде. В ходе разработки приложения TaskManager применялись следующие виды и методы отладки.

Инструменты отладки.

Основным инструментом отладки служил встроенный отладчик Microsoft Visual Studio 2022, предоставляющий следующие возможности:

- точки останова (Breakpoints) - позволяют приостановить выполнение программы в заданном месте кода для анализа состояния переменных и объектов.
- условные точки останова (Conditional Breakpoints) - срабатывают только при выполнении заданного условия, что особенно полезно при отладке циклов и коллекций.
- окно «Locals» - отображает значения всех локальных переменных в текущем стеке вызовов.
- окно «Watch» - позволяет отслеживать значения конкретных выражений и переменных в процессе выполнения.
- окно «Call Stack» - отображает стек вызовов, позволяя отследить путь выполнения кода.
- пошаговое выполнение (Step Into, Step Over, Step Out) - позволяет выполнять код построчно, входить в вызываемые методы или выходить из них.
- окно «Immediate Window» - позволяет выполнять произвольные выражения C# в контексте текущей точки останова.

Процесс проведения отладки приложения TaskManager.

В ходе разработки было выявлено и устранено несколько категорий ошибок:

- ошибки логики в методе Search() класса TaskService - при выполнении поиска по пустой строке метод должен возвращать все задачи после отладки добавлена проверка `string.IsNullOrEmpty(query)`
- ошибки сериализации - при первоначальной реализации перечисления TaskStatus и TaskPriority сериализовались как числа после добавления `JsonStringEnumConverter` они корректно преобразуются в строковые представления.
- ошибки в методе Delete() - при удалении несуществующей задачи требовалось выбрасывать `KeyNotFoundException` вместо молчаливого игнорирования операции после отладки добавлена соответствующая проверка.
- ошибки привязки данных в WPF - при изменении свойств `TaskItemViewModel` не происходило обновление интерфейса проблема решена корректной реализацией `INotifyPropertyChanged` в базовом классе `BaseViewModel`.

Результаты отладки:

По результатам отладки все выявленные ошибки были устранены. Приложение корректно обрабатывает граничные случаи: пустые строки в поиске, попытки удаления несуществующих задач, загрузку несуществующего файла. Для всех критических операций реализована обработка исключений с информативными сообщениями об ошибках.

РАЗДЕЛ 4. ОСУЩЕСТВЛЯТЬ РАЗРАБОТКУ ТЕСТОВЫХ НАБОРОВ И ТЕСТОВЫХ СЦЕНАРИЕВ ДЛЯ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

4.1. Тестирование

Описание процесса проведения тестирования ПО:

Тестирование программного обеспечения - процесс верификации и валидации программного продукта с целью обнаружения дефектов и подтверждения соответствия требованиям. В проекте TaskManager применялись следующие виды тестирования:

- модульное тестирование (Unit Testing) - тестирование отдельных классов и методов в изоляции от остальной системы реализовано с использованием фреймворка xUnit в проекте TaskManager.Tests
- интеграционное тестирование - проверка взаимодействия компонентов системы, в частности взаимодействия TaskService с JsonTaskRepository.
- функциональное тестирование - ручная проверка соответствия приложения функциональным требованиям технического задания
- тестирование граничных значений - проверка поведения системы при граничных условиях (пустые строки, отсутствующие идентификаторы, пустые коллекции).

Тестовые сценарии:

В Таблица 4 - Тестирование ниже представлены тестовые сценарии для основных функций приложения TaskManager.

Таблица 4 - Тестирование

№	Тестируемая функция	Входные данные	Ожидаемый результат	Статус
1	Добавление задачи	Корректный объект TaskItem	Задача добавлена в список	Пройден
2	Добавление задачи с пустым названием	TaskItem с Title = ""	ArgumentException	Пройден
3	Получение задачи по Id	Существующий Guid	Объект TaskItem	Пройден
4	Получение задачи по несуществующему Id	Несуществующий Guid	null	Пройден
5	Удаление задачи	Существующий Guid	Задача удалена из списка	Пройден
6	Удаление несуществующей задачи	Несуществующий Guid	KeyNotFoundException	Пройден
7	Обновление задачи	Измененный TaskItem	Задача обновлена	Пройден
8	Фильтрация по статусу	TaskStatus.Completed	Только завершенные задачи	Пройден
9	Поиск по названию	Строка запроса	Задачи с совпадением в названии	Пройден
10	Поиск пустой строкой	""	Все задачи	Пройден
11	Сортировка по приоритету	descending = true	Задачи в убывающем порядке приоритета	Пройден
12	Сортировка по сроку	ascending = true	Задачи в возрастающем порядке даты	Пройден
13	Статистика	Набор задач разных статусов	Корректные счетчики	Пройден
14	Сохранение в файл	Путь к файлу	JSON-файл создан и содержит данные	Пройден
15	Загрузка из файла	Существующий JSON-файл	Задачи загружены в список	Пройден

4.2. Unit-тесты

Модульные тесты реализованы в проекте TaskManager.Tests с использованием фреймворка xUnit. Фреймворк xUnit выбран как современный, широко используемый в .NET-экосистеме инструмент с поддержкой параметризованных тестов и удобной интеграцией с Visual Studio Test Explorer.

Ниже представлены итог unit-тестов, покрывающих ключевые сценарии работы приложения Рисунок 4.2.1 – Итог тестирования:

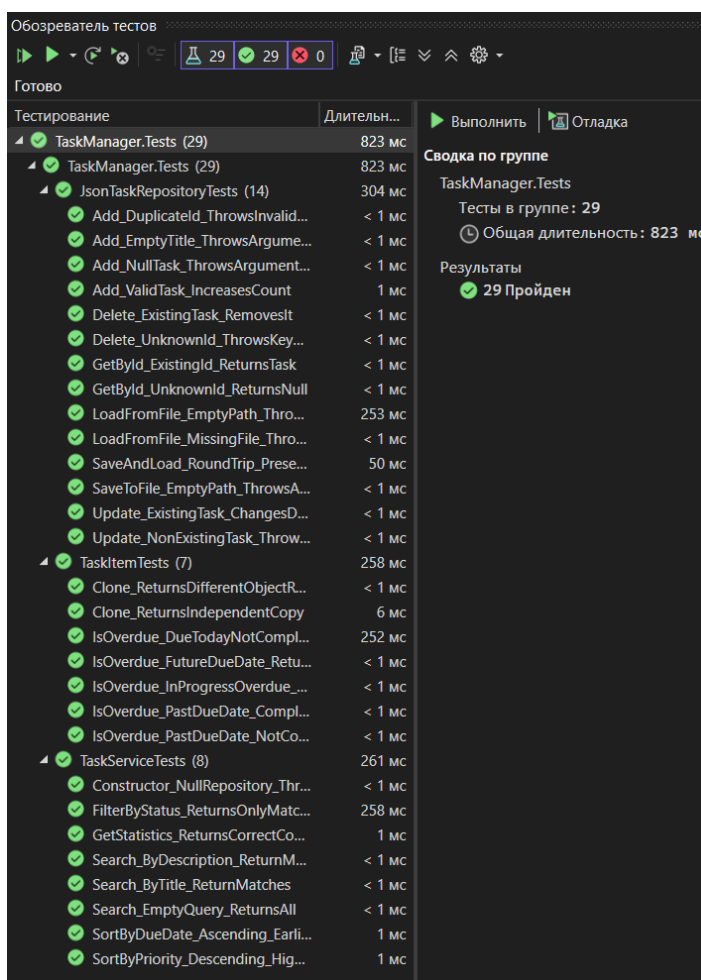


Рисунок 4.2.1 – Итог тестирования

Всего в проекте TaskManager.Tests реализовано 29 unit-тестов, покрывающих все публичные методы классов JsonTaskRepository и TaskService. Все тесты проходят успешно.

РАЗДЕЛ 5. ИНСПЕКТИРОВАНИЕ КОМПОНЕНТ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ НА ПРЕДМЕТ СООТВЕТСТВИЯ СТАНДАРТАМ КОДИРОВАНИЯ

5.1. Процесс инспектирования компонент программного обеспечения на предмет соответствия стандартам кодирования

Виды инспекции программного кода:

Инспектирование программного обеспечения - формальный процесс проверки кода с целью обнаружения дефектов, обеспечения соответствия стандартам кодирования и повышения качества программного продукта. В ходе разработки приложения TaskManager применялись следующие виды инспекции:

- статический анализ кода - автоматическая проверка исходного кода без его выполнения с использованием специализированных инструментов позволяет выявить потенциальные ошибки, нарушения стандартов и уязвимости безопасности.

- ручная инспекция кода (Code Review) - систематический просмотр исходного кода разработчиком с целью обнаружения логических ошибок, нарушений стиля кодирования и несоответствий архитектурным решениям.

- инспекция на основе контрольных списков (Checklist-Based Inspection) - проверка кода по заранее составленному перечню критериев, охватывающих типичные ошибки и требования стандартов.

Актуальные стандарты кодирования C#.

При разработке приложения TaskManager соблюдались следующие стандарты и соглашения о кодировании:

- Microsoft C# Coding Conventions - официальные соглашения Microsoft по именованию, форматированию и стилю кода на C#: PascalCase для публичных членов классов и типов, camelCase для локальных переменных и параметров методов, префикс подчеркивания (_) для приватных полей.

– SOLID-принципы - применение принципов проектирования: Single Responsibility (каждый класс имеет единственную ответственность), Open/Closed (классы открыты для расширения, закрыты для модификации), Interface Segregation (интерфейсы ITaskRepository и ITaskService содержат только необходимые методы).

– Clean Code (Роберт Мартин) - осмысленные имена переменных, методов и классов, небольшие методы, выполняющие одну функцию и минимизацию вложенности.

– XML-документация - использование тегов <summary> для документирования публичных методов и классов.

– Принцип DRY (Don't Repeat Yourself) - отсутствие дублирования кода, повторно используемая логика вынесена в отдельные методы.

Программы для осуществления инспекции.

Для инспектирования кода приложения TaskManager использовались следующие инструменты их можно увидеть в Таблица 5 - Инструменты

Таблица 5 - Инструменты

Инструмент	Назначение	Результат проверки
Visual Studio Code Analysis	Встроенный анализатор кода Visual Studio, выявляющий предупреждения компилятора и нарушения правил	Все предупреждения устранены
Roslyn Analyzers (.NET)	Анализаторы на базе Roslyn - платформы компилятора .NET, выявляющие паттерны нарушений и стилевые проблемы	Замечания устранены
SonarLint for Visual Studio	Расширение для статического анализа кода, выявляющее потенциальные ошибки, уязвимости.	Критических проблем не выявлено

Результаты инспектирования кода TaskManager.

В ходе инспектирования кода приложения TaskManager были выявлены и устранены следующие замечания:

- в первоначальной версии метода Delete() отсутствовала проверка наличия задачи перед удалением, после инспекции добавлена проверка с выбрасыванием KeyNotFoundException.

- метод ReplaceAll() в классе JsonTaskRepository не был объявлен в интерфейсе ITaskRepository, что нарушало принцип инкапсуляции, после анализа принято решение оставить метод в конкретном классе и вызывать его только через TaskService.RestoreSnapshot().

- в ряде методов TaskService отсутствовала проверка null-аргументов, после инспекции добавлены соответствующие проверки с ArgumentNullException.

По итогам инспектирования код приложения TaskManager приведен в соответствие с актуальными стандартами кодирования C# и принципами объектно-ориентированного проектирования. Все критические замечания устранены, код документирован и соответствует требованиям технического задания.

ЗАКЛЮЧЕНИЕ

В результате прохождения учебной практики с 8 по 28 июня 2026 г. были закреплены теоретические знания и расширены профессиональные умения.

По итогам практики получены следующие результаты:

- разработано приложение для управления задачами TaskManager, реализующее полный набор требуемых функций: добавление, просмотр, редактирование, удаление задач, фильтрацию по статусу, поиск, сортировку, подсчет статистики, сохранение и загрузку данных в формате JSON.
- архитектура приложения реализована в соответствии с принципами SOLID и паттерном MVVM: бизнес-логика вынесена в динамическую библиотеку TaskManager.Core, пользовательский интерфейс реализован в проекте TaskManager.WPF с применением механизма привязки данных WPF.
- написан комплект модульных тестов (29 тестов) на базе фреймворка xUnit, покрывающих все ключевые сценарии работы классов JsonTaskRepository и TaskService.
- проведена отладка приложения с использованием встроенного отладчика Visual Studio 2022, выявленные дефекты устранены.
- выполнено инспектирование кода на предмет соответствия стандартам кодирования C# Microsoft Coding Conventions - все замечания устранены.

По окончании практики был получен практический опыт:

- разработки требований к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент.
- интеграции модулей в программное обеспечение с использованием трёхуровневой архитектуры.
- отладки программного модуля с использованием специализированных программных средств (Visual Studio Debugger).

- разработки тестовых наборов и тестовых сценариев для программного обеспечения с применением фреймворка xUnit.
- инспектирования компонент программного обеспечения на предмет соответствия стандартам кодирования с использованием статических анализаторов кода.

Разработанное приложение TaskManager соответствует всем требованиям технического задания и демонстрирует практическое применение ключевых технологий и подходов современной разработки программного обеспечения на C# и .NET 8.

По окончании практики был получен практический опыт разработки требования к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент, интеграции модулей в программное обеспечение, отладки программного модуля с использованием специализированных программных средств, разработки тестовых наборов и тестовых сценариев для программного обеспечения, инспектирования компонент программного обеспечения на предмет соответствия стандартам кодирования.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Microsoft. C# Documentation. [Электронный ресурс]. - Режим доступа: <https://docs.microsoft.com/ru-ru/dotnet/csharp/> (дата обращения: 08.06.2026).
2. Microsoft. WPF Documentation. [Электронный ресурс]. - Режим доступа: <https://docs.microsoft.com/ru-ru/dotnet/desktop/wpf/> (дата обращения: 10.06.2026).
3. Мартин Р. Чистый код: создание, анализ и рефакторинг. - СПб.: Питер, 2019. - 464 с.
4. Фаулер М. Архитектура корпоративных программных приложений. - М.: Вильямс, 2016. - 544 с.
5. ГОСТ 19.201-78. Техническое задание. Требования к содержанию и оформлению. - М.: Издательство стандартов, 1978.
6. ГОСТ 34.602-89. Техническое задание на создание автоматизированной системы. - М.: Издательство стандартов, 1989.
7. Microsoft. .NET 8 Documentation. [Электронный ресурс]. - Режим доступа: <https://docs.microsoft.com/ru-ru/dotnet/core/whats-new/dotnet-8> (дата обращения: 10.06.2026).
8. xUnit.net Documentation. [Электронный ресурс]. - Режим доступа: <https://xunit.net/> (дата обращения: 15.06.2026).
9. Microsoft. System.Text.Json Documentation. [Электронный ресурс]. - Режим доступа: <https://docs.microsoft.com/ru-ru/dotnet/standard/serialization/system-text-json-overview> (дата обращения: 10.06.2026).
10. Microsoft. MVVM Pattern. [Электронный ресурс]. - Режим доступа: <https://docs.microsoft.com/ru-ru/dotnet/architecture/maui/mvvm> (дата обращения: 10.06.2026).

Требование к написанию программного модуля

Задание.

Цель:

Практика работы с классами, коллекциями, файловым вводом-выводом, LINQ и основами ООП в C#.

1. Основные требования: Разработать приложение для управления задачами с возможностью:

1.1 Добавления новой задачи (название, описание, приоритет, срок выполнения, статус).

1.2 Просмотра списка всех задач.

1.3 Фильтрации задач по статусу (например, "Новая", "В процессе", "Завершена").

1.4 Поиска задач по названию или описанию.

1.5 Редактирования существующих задач.

1.6 Удаления задач.

1.7 Сохранения и загрузки задач в/из файла (JSON или XML).

1.8 Основная логика приложения должна находиться в динамической библиотеке, а пользовательский интерфейс должен представлять из себя WPF-приложение.

1.9 Для классов в библиотеке необходимо написать модульные тесты.

2. Дополнительные задания (по желанию)

2.1 Реализовать сортировку задач по приоритету или сроку выполнения.

2.2 Добавить возможность пометки задачи как "Важная" (выводить звёздочкой или цветом в консоли).

2.3 Реализовать простую статистику (например, сколько задач завершено, сколько просрочено).

Сопроводительная записка должна содержать:

- титульный лист,

- оглавление,
- диаграмму классов,
- подробное описание действий по созданию ПО (с иллюстрациями).

Исходные коды приложения и сопроводительную записку выложить на GitHub и ссылку на репозиторий прислать мне на почту.